

IC526 Advanced Compilers

Lec 11: LLVM IR II

Yoonseung Kim

DGIST EECS

Fall 2026

Announcement

- PA 3 will be posted this week.
 - Deadline: after the midterm week

Recap: LLVM IR, without Control Flow

LLVM IR I: functions, typed values, and memory

```
%x.addr = alloca i32  
store i32 10, ptr %x.addr  
%x = load i32, ptr %x.addr
```

%x.addr is an address. %x is the stored integer value.

LLVM IR II: control flow, arrays, and reference parameters

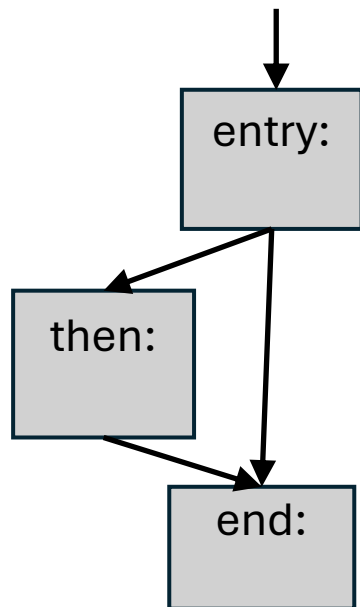
Today's Topics

- Control flow: a directed graph of basic blocks
- Translation of `if` and `while` statements
- Arrays
- Using pointers to represent references and forwarding

Control Flow

Basic Blocks

- A function body is a **directed graph** of basic blocks.
- A basic block is a (straight-line) sequence of instructions.
 - It ends with a terminator – a special instruction that determines what happens next.



```
entry:
    %next = add i32 %x, 1
    %test = icmp slt i32 %next, 10
    br i1 %test, label %then, label %end

then:
    ...
```

Terminator Instructions

- Unconditional branch

`br label %next`

- Conditional branch

`br i1 %cond, label %then, label %else`

- The condition has type `i1`.
- Both destinations are blocks in this function.

- Return

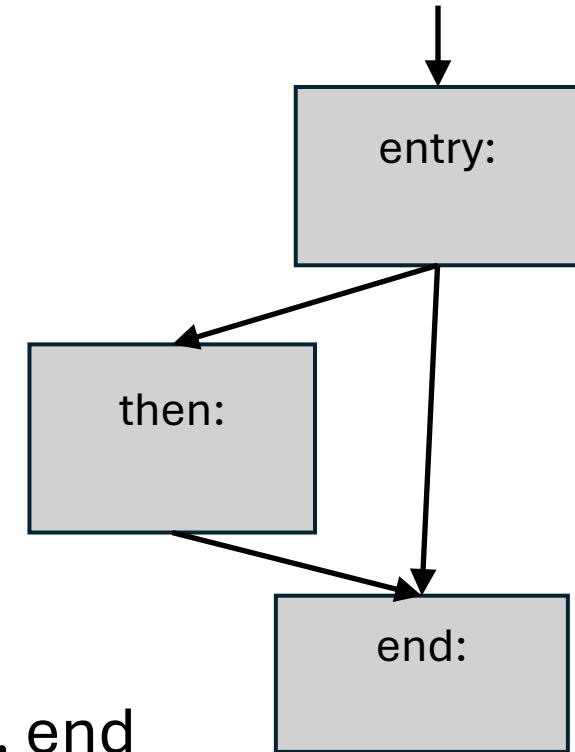
`ret i32 %x`

`ret void`

- Others: `switch`, `indirectbr`, `callbr`, `unreachable`, ...

Control-Flow Graph (CFG)

- The directed graph of basic blocks
 - Nodes: basic blocks
 - Edges: possible transfers of control
- The first block is the entry block.
- The entry block has no predecessors.

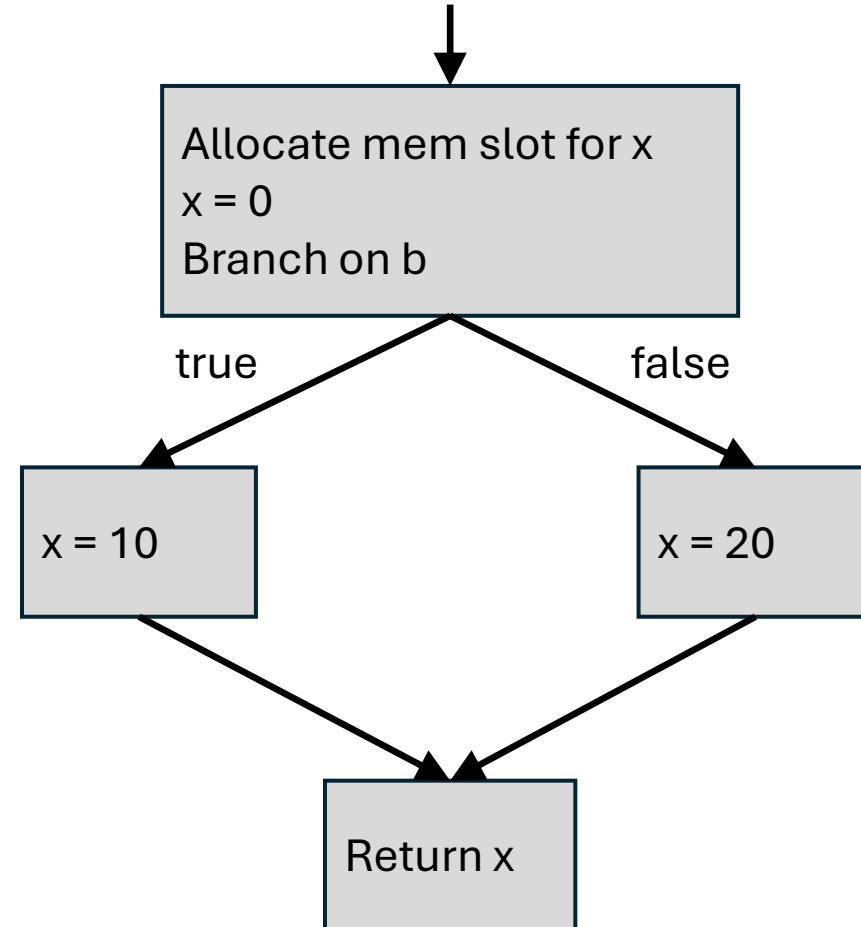


Successors of entry: then, end
Predecessors of then: entry

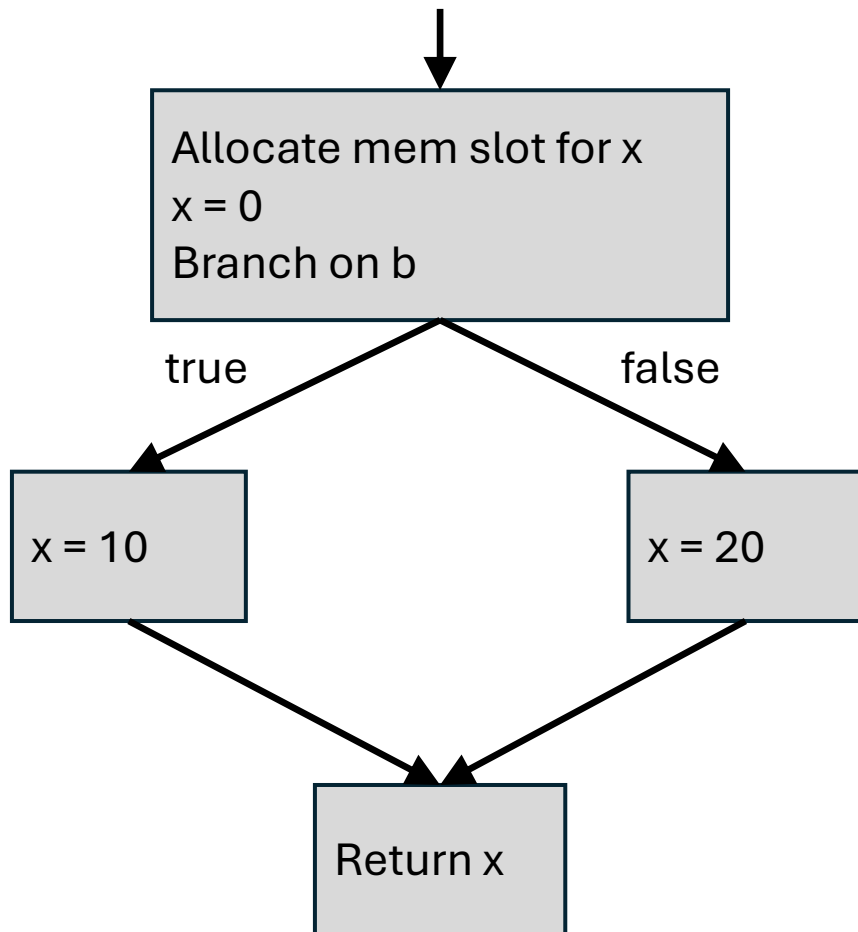
An If Statement

(Your translation does not need to match this example exactly.)

```
fn choose(b: bool) -> i32 {  
    let mut x = 0;  
    if b {  
        x = 10;  
    } else {  
        x = 20;  
    }  
    return x;  
}
```



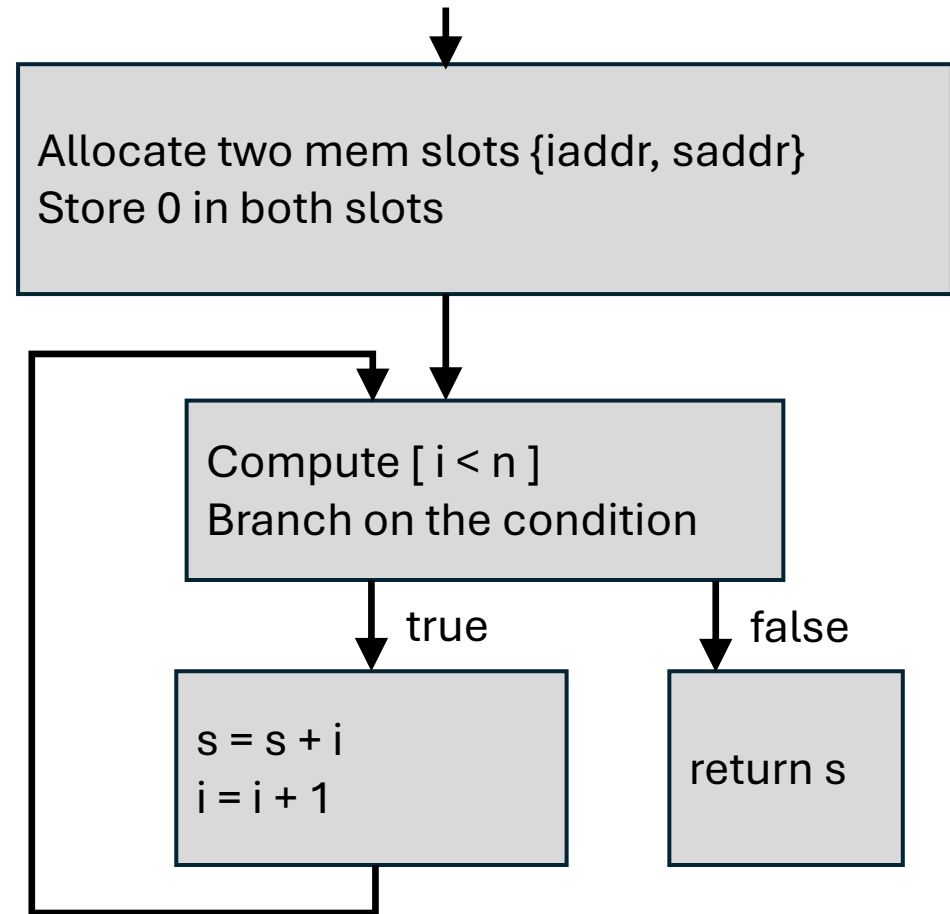
An If Statement, Translated



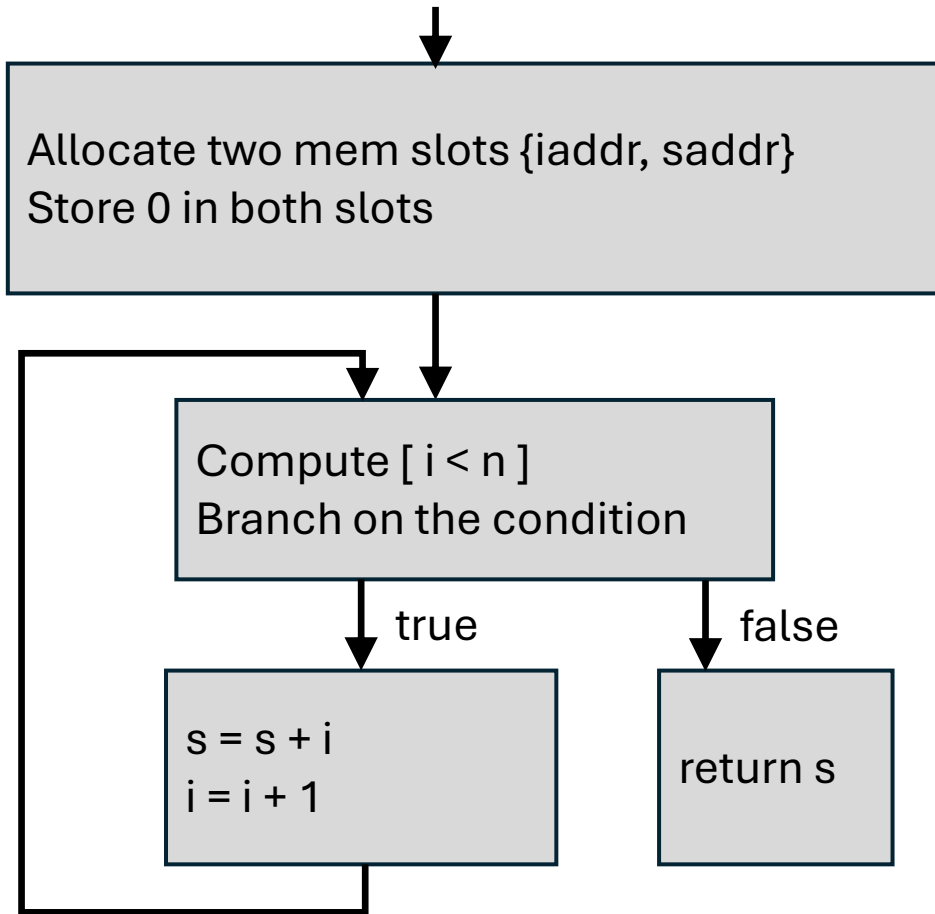
```
define i32 @choose(i1 %b) {  
entry:  
    %x.addr = alloca i32  
    store i32 0, ptr %x.addr  
    br i1 %b, label %then, label %else  
then:  
    store i32 10, ptr %x.addr  
    br label %merge  
else:  
    store i32 20, ptr %x.addr  
    br label %merge  
merge:  
    %result = load i32, ptr %x.addr  
    ret i32 %result  
}
```

A While Statement

```
fn sum(n: usize) -> usize {  
    let mut i: usize = 0;  
    let mut s: usize = 0;  
    while i < n {  
        s = s + i;  
        i = i + 1;  
    }  
    return s;  
}
```

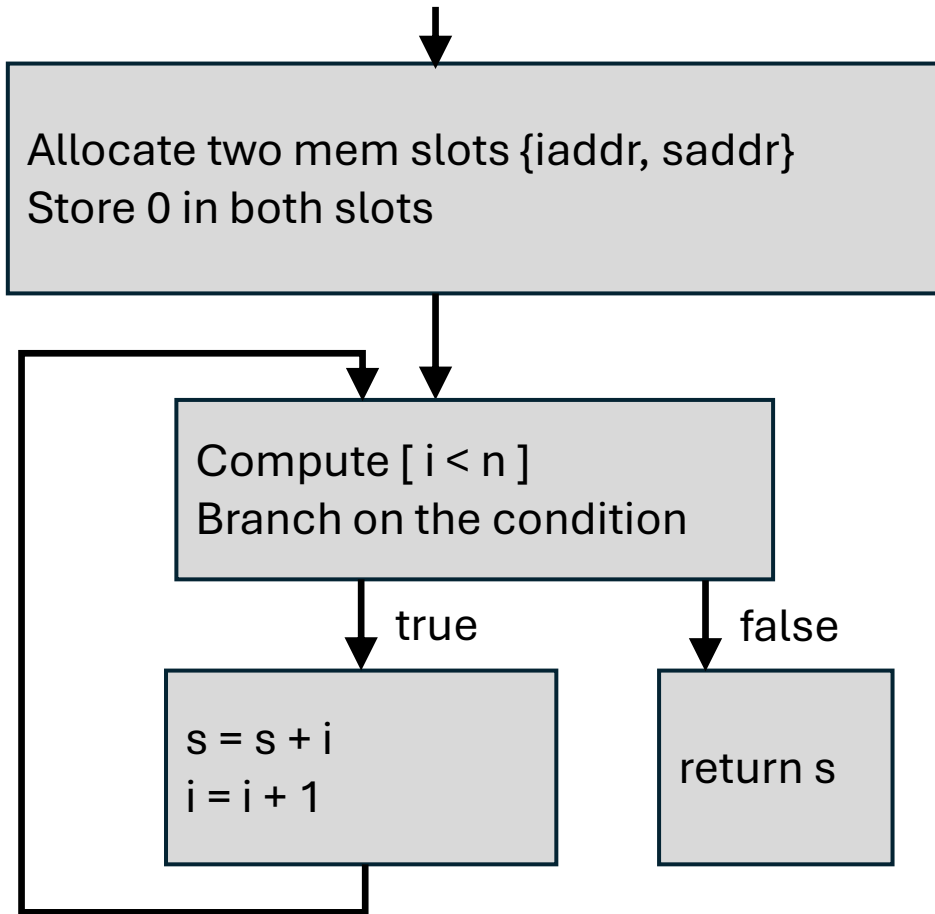


A While Statement, Translated



```
define i64 @sum(i64 %n) {  
entry:  
    %i.addr = alloca i64  
    %s.addr = alloca i64  
    store i64 0, ptr %i.addr  
    store i64 0, ptr %s.addr  
    br label %cond  
cond:  
    %iv = load i64, ptr %i.addr  
    %test = icmp ult i64 %iv, %n  
    br i1 %test, label %body, label %exit
```

A While Statement, Translated (cont.)



cond:

```
%iv = load i64, ptr %i.addr  
%test = icmp ult i64 %iv, %n  
br i1 %test, label %body, label %exit
```

body:

```
%sv = load i64, ptr %s.addr  
%sum = add i64 %sv, %iv  
store i64 %sum, ptr %s.addr  
%next = add i64 %iv, 1  
store i64 %next, ptr %i.addr  
br label %cond
```

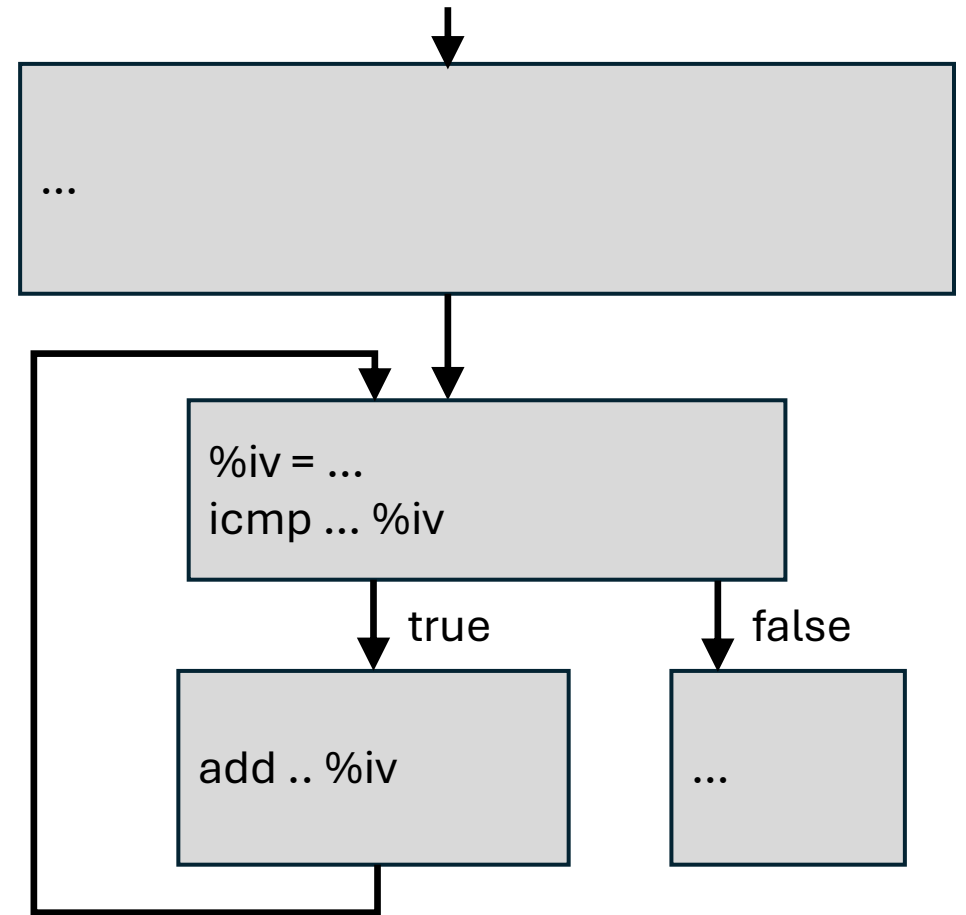
exit:

```
%result = load i64, ptr %s.addr  
ret i64 %result
```

}

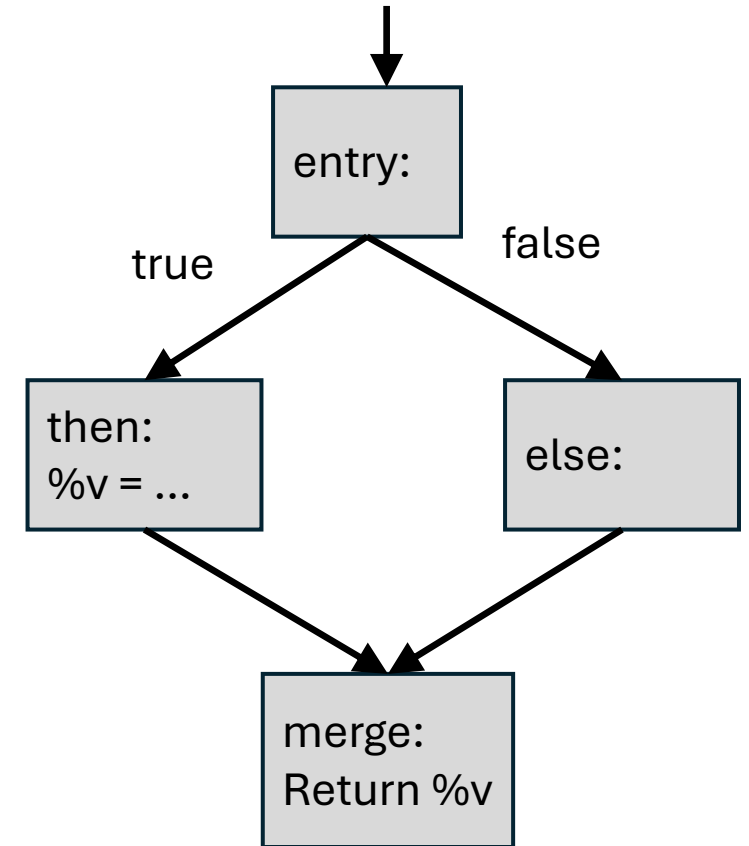
Dominance

- Block A dominates block B if every path from the entry block to B passes through A.
- An ordinary use of an SSA value must be dominated by its definition.
- Within one block, define a value before using it.
- Function parameters are available throughout the body.



A Value Defined in Only One Branch

```
entry:
    br i1 %c, label %then, label %else
then:
    %v = add i32 %a, 1
    br label %merge
else:
    br label %merge
merge:
    ret i32 %v                ; invalid
```



The path through `else` never defines `%v`.

After the midterm, we will revisit this topic with phi nodes.

Arrays

Arrays in Memory

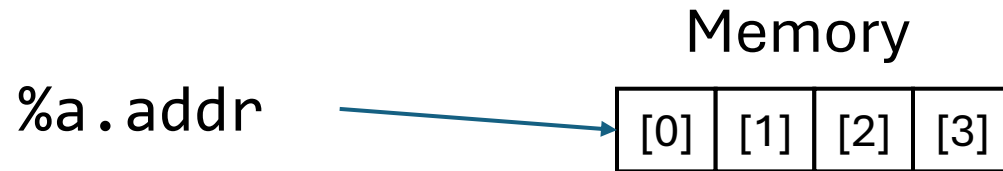
```
let a: [i32; 4] = [10, 20, 30, 40];
```



```
%a.addr = alloca [4 x i32]  
store [4 x i32] [i32 10, i32 20, i32 30, i32 40], ptr %a.addr
```

The store writes the whole array **constant**.

GetElementPtr: For Addresses of Elements



```
%elem.addr = getelementptr [4 x i32], ptr %a.addr,  
                           i64 0, i64 %i
```

```
%v = load i32, ptr %elem.addr
```

getelementptr (GEP) computes an address without accessing memory.

$$\%a.addr + 0 * \text{sizeof}([4 \times i32]) + \%i * \text{sizeof}(i32) = \%a.addr + \%i * 4$$

Equivalent: `%elem.addr = getelementptr i32, ptr %a.addr, i64 %i`

Note: GEP performs no bounds check.

Reading and Writing an Element

```
let mut a: [i32; 4] = [10, 20, 30, 40];  
let x = a[2];  
a[2] = x + 1;
```



```
%elem.addr = getelementptr [4 x i32], ptr %a.addr,  
                           i64 0, i64 2  
%x = load i32, ptr %elem.addr  
%next = add i32 %x, 1  
store i32 %next, ptr %elem.addr
```

Constructing a Non-Constant Array Value

```
let a: [i32; 2] = [x, y];
```



```
%a.addr = alloca [2 x i32]  
%a0 = insertvalue [2 x i32] undef, i32 %x, 0  
%a1 = insertvalue [2 x i32] %a0, i32 %y, 1  
store [2 x i32] %a1, ptr %a.addr
```

undef	undef
-------	-------

%x	undef
----	-------

%x	%y
----	----

- `insertvalue` produces a new array value with one element replaced.
- It uses a constant index and does not write memory.
- Fill every element, then store the array.

References and noalias

Reference Parameters

RustY reference types are translated into the LLVM pointer type.

RustY parameter	LLVM type
<code>&T</code>	<code>ptr</code>
<code>&mut T</code>	<code>ptr</code>

- `ptr` does not encode the pointee type or its mutability.
- Our compiler adds the `noalias` attribute to `&mut` parameters.

Dereferencing a Parameter

```
fn add_to(p: &mut i32, delta: i32) {  
    *p = *p + delta;  
}
```



```
define void @add_to(ptr noalias %p, i32 %delta) {  
entry:  
    %old = load i32, ptr %p  
    %new = add i32 %old, %delta  
    store i32 %new, ptr %p  
    ret void  
}
```

%p holds the address of the referenced object.

Creating a Reference Argument

```
let mut n: i32 = 0;  
add_to(&mut n, 2);
```



```
%n.addr = alloca i32  
store i32 0, ptr %n.addr  
call void @add_to(ptr %n.addr, i32 2)
```

&mut n passes the address of n's storage.
For an &T parameter, &n also passes that address.

Forwarding an Existing Reference

```
fn twice(p: &mut i32) {  
    add_to(p, 1);  
    add_to(p, 1);  
}
```



```
define void @twice(ptr noalias %p) {  
entry:  
    call void @add_to(ptr %p, i32 1)  
    call void @add_to(ptr %p, i32 1)  
    ret void  
}
```

Forwarding passes the received pointer to another function.

A Reference to an Array

```
fn get(a: &[i32; 4], i: usize) -> i32 {  
    return (*a)[i];  
}
```



```
define i32 @get(ptr %a, i64 %i) {  
entry:  
    %elem.addr = getelementptr [4 x i32], ptr %a,  
                                i64 0, i64 %i  
    %v = load i32, ptr %elem.addr  
    ret i32 %v  
}
```

The source array type supplies [4 x i32] to GEP.

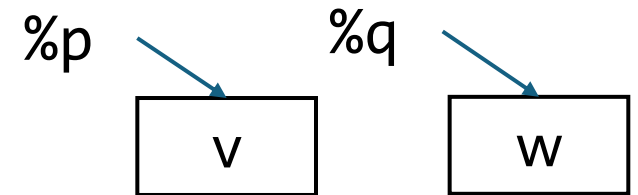
noalias: Can a Store Change a Later Load?

```
fn observe(p : &mut i32, q : &i32) -> i32 {  
    let before = *q;  
    *p = 7;  
    let after = *q;  
    return before + after;  
}
```

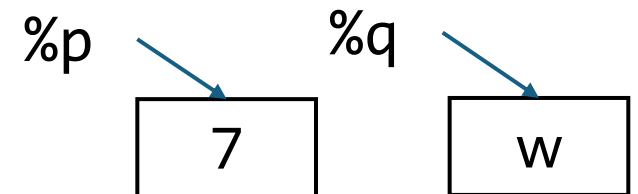


```
define i32 @observe(ptr %p, ptr %q) {  
entry:  
    %before = load i32, ptr %q  
    store i32 7, ptr %p  
    %after = load i32, ptr %q  
    %sum = add i32 %before, %after  
    ret i32 %sum  
}
```

Can we replace %after with %before?



%before = w

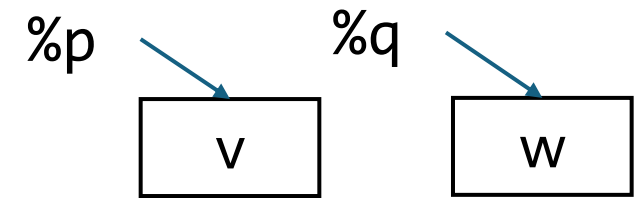


%after = w

%sum = w + w

noalias Enables Load Reuse

```
define i32 @observe(ptr noalias %p, ptr %q) {  
entry:  
    %before = load i32, ptr %q  
    store i32 7, ptr %p  
    %sum = add i32 %before, %before  
    ret i32 %sum  
}
```



The store through `%p` cannot change the value read through `%q`.

RustY's Borrow Contract

```
fn observe(p: &mut i32, q: &i32) -> i32 {  
    let before = *q;  
    *p = 7;  
    let after = *q;  
    return before + after;  
}
```

```
observe(&mut x, &x);    // rejected
```

- Our compiler adds `noalias` to `&mut` parameters.
- LLVM uses the promise without a runtime alias check.
- Violating the `noalias` contract causes undefined behavior.

Summary

- Branches connect basic blocks into a CFG.
- GEP computes an element address.
- References and forwarding pass pointer values.
- noalias communicates a memory-access contract.

Next: generate this IR from RustY's typed AST.